

NLP Lab — Stage 1: Data Acquisition (Beginner Guide)

B.Tech CS&E · Semester VII · Course 2604732 · GLS FET

Where are we in the NLP pipeline?

```
>> STAGE 1: DATA ACQUISITION << → Stage 2: Preprocessing → Stage 3: Vectorization → Stage 4: Mode
1
      (get the text into Python)      (the 13 cleaning steps      (turn words into numbers)
                                     you already learned)
```

We did the *cleaning* first in the lab because it's hands-on. But in a real project, **getting the text comes first** — you can't clean text you haven't loaded yet. So this notebook is really **Step 0**.

The ONE idea to remember

No matter where text comes from — something you type, a file, a website — it always ends up as a **Python string** (or a **list of strings**). Once it's a string, it flows straight into the 13-step preprocessing pipeline. **That's the whole job of Stage 1: turn any source into text Python can hold.**

This notebook **creates its own sample files**, so you don't need to download anything to follow along.

0. Setup

Run this once. It installs the libraries and makes a small `sample_data/` folder with files we'll read later.

```
In [1]: !pip install pandas nltk beautifulsoup4 requests -q

import os, json
import pandas as pd

os.makedirs("sample_data", exist_ok=True)

# a plain text file
with open("sample_data/note.txt", "w", encoding="utf-8") as f:
    f.write("Natural Language Processing is fun.\nText data is everywhere!")

# a CSV file of product reviews (text + a label)
pd.DataFrame({
    "review": ["Loved this product!", "Terrible, waste of money", "It was okay"],
    "label": ["positive", "negative", "neutral"]
}).to_csv("sample_data/reviews.csv", index=False)

# a JSON file of tweets
json.dump(
    [{"user": "asha", "text": "NLP is so cool"},
    {"user": "ravi", "text": "this assignment is hard"}],
    open("sample_data/tweets.json", "w"))

print("Ready. Files created:", os.listdir("sample_data"))
```

Ready. Files created: ['note.txt', 'reviews.csv', 'tweets.json']

Way 1 — Type the text yourself (a string)

What it is: the simplest source — text written directly in your code. **When you use it:** to quickly test an idea before bringing in real data.

A single piece of text is a **string**. Many pieces of text are a **list of strings** — this is the shape NLP usually wants (one string per document).

```
In [7]: # ONE document = a string
text = "Natural Language Processing is fun!"
print(text)
print("This is a:", type(text).__name__)

# MANY documents = a list of strings
docs = ["great product, loved it",      # document 0
        "terrible, waste of money",    # document 1
        "it was okay"]                # document 2

print("\nWe have", len(docs), "documents:")
for i, d in enumerate(docs):
    print(" doc", i, "->", d)
```

```
Natural Language Processing is fun!
This is a: str
```

```
We have 3 documents:
doc 0 -> great product, loved it
doc 1 -> terrible, waste of money
doc 2 -> it was okay
```

You'll see: the single string printed, and the three documents listed with their position numbers. *This docs list is exactly what Stage 2 cleans.*

Way 2 — Read a text file (.txt)

What it is: loading text that's saved in a file on disk. **When you use it:** the text is in a .txt (a book, an article, notes).

`open(...).read()` reads the **whole file into one string**. Always add `encoding="utf-8"` so symbols and other languages load correctly.

```
In [3]: # the "r" means read mode
with open("sample_data/note.txt", "r", encoding="utf-8") as f:
    text = f.read()

print(text)
print("\nTotal characters:", len(text))

# if you want each line separately:
with open("sample_data/note.txt", "r", encoding="utf-8") as f:
    lines = f.read().splitlines()
print("As a list of lines:", lines)
```

Natural Language Processing is fun.
Text data is everywhere!

Total characters: 60
As a list of lines: ['Natural Language Processing is fun.', 'Text data is everywhere!']

You'll see: the file's two lines printed, the character count, then the same text split into a list of lines. **Beginner tip:** the file must be in the same folder, or give the full path — `FileNotFoundError` almost always means a wrong path.

Way 3 — Read a CSV file (with pandas) ★ *the most common one*

What it is: reading a spreadsheet-like file where each row is one document. **When you use it:** almost all real NLP datasets (reviews, tweets, spam) come as CSV.

A CSV usually has a **text column** and a **label column** (the answer). `pandas` reads it into a table called a **DataFrame**; we pull the text column out as a list with `.tolist()`.

```
In [ ]: df = pd.read_csv("sample_data/reviews.csv")
print(df)                                # show the whole table

# pull out just the text column as a list of strings
texts = df["review"].tolist()
labels = df["label"].tolist()

print("\nTexts :", texts)
print("Labels:", labels)
```

You'll see: the 3-row table, then the review texts and their labels as two lists. The `texts` list is what goes into preprocessing; the `labels` list is what a model will later learn to predict.

Way 4 — Read a JSON file

What it is: reading text stored in JSON — a format of `{ key: value }` pairs. **When you use it:** most websites and APIs (Twitter/X, Reddit) return data as JSON.

JSON loads into Python **lists and dictionaries**. You pick out the field you need (here, `"text"`).

```
In [ ]: with open("sample_data/tweets.json", "r", encoding="utf-8") as f:
        data = json.load(f)

print("Full data:", data)

# keep only the text of each tweet
tweet_texts = [item["text"] for item in data]
print("\nJust the text:", tweet_texts)
```

You'll see: the full list of tweet dictionaries, then just the tweet texts pulled out — again, a list of strings ready for Stage 2.

Way 5 — Use a built-in dataset ★ *best for practice*

What it is: ready-made, clean, labelled text that ships with libraries. **When you use it:** for learning and assignments — you skip the messy data-cleaning of finding/loading data and focus on NLP itself.

NLTK includes several. `movie_reviews` is great because it's already labelled **positive / negative** — perfect for a first sentiment project.

```
In [8]: import nltk
nltk.download("movie_reviews", quiet=True)
from nltk.corpus import movie_reviews

print("Labels available:", movie_reviews.categories())
print("Total reviews  :", len(movie_reviews.fileids()))

# grab one negative and one positive review
neg_id = movie_reviews.fileids("neg")[0]
one_review = movie_reviews.raw(neg_id)
print("\nFirst 200 characters of a NEGATIVE review:\n", one_review[:200])
```

```
Labels available: ['neg', 'pos']
Total reviews   : 2000
```

First 200 characters of a NEGATIVE review:

```
plot : two teen couples go to a church party , drink and then drive .
they get into an accident .
one of the guys dies , but his girlfriend continues to see him in her life , and has nightmares .
w
```

You'll see: the two labels (`neg` , `pos`), a count of 2000 reviews, and the start of one review. **Note:** this cell downloads a small dataset, so it needs internet the first time (just like the `punkt_tab` download in your preprocessing lab).

Way 6 — Get text from a website (web scraping) (*good to know*)

What it is: downloading a web page and pulling the readable text out of the HTML. **When you use it:** the text you want is on a website with no file or dataset.

This is **exactly why preprocessing has a "remove HTML tags" step** — scraped text arrives full of tags.

First the **no-internet version** — clean an HTML string directly:

```
In [9]: from bs4 import BeautifulSoup

html = "<html><body><h1>NLP News</h1><p>A <b>new</b> model was released today.</p></body></html>"

# get_text() strips the tags; separator=" " keeps words from sticking together
clean_text = BeautifulSoup(html, "html.parser").get_text(separator=" ")
print(clean_text)
```

NLP News A new model was released today.

And the **live version** (needs internet). It's wrapped in `try/except` so the lab won't crash if there's no connection — a good habit for any code that touches the internet.

```
In [12]: import requests
from bs4 import BeautifulSoup

url = "https://en.wikipedia.org/wiki/Natural_language_processing"
try:
    html = requests.get(url, timeout=10).text          # download the page
    soup = BeautifulSoup(html, "html.parser")         # parse the HTML
    paragraphs = [p.get_text() for p in soup.find_all("p")]
    article = " ".join(paragraphs)
    print("Downloaded", len(article), "characters")
    print("\nFirst 250:\n", article[:250])
except Exception as e:
    print("No internet right now – that's fine for the lab. Details:", e)
```

Downloaded 0 characters

First 250:

Quick summary — which source, which tool

Source	How to load it	You get
Typed text	<code>text = "..."</code> / <code>["...", "..."]</code>	string / list of strings
.txt file	<code>open(path).read()</code>	one string
.csv file	<code>pd.read_csv(path) → df["col"].tolist()</code>	list of strings
.json file	<code>json.load(f) → pick the field</code>	list of strings
Built-in dataset	<code>from nltk.corpus import ...</code>	ready labelled text
Website	<code>requests + BeautifulSoup</code>	one string (tags removed)

Every row ends in the same place: text Python can hold.

The bridge — Stage 1 hands off to Stage 2

Whatever the source, we now have text — so it flows straight into the **13-step preprocessing pipeline** you already built (lowercase, remove HTML/URLs, tokenize, stopwords, lemmatize, ...).


```
In [13]: # pretend these came from three different sources above
collected = [
    "Natural Language Processing is fun!", # typed string
    "Loved this product!",               # from the CSV
    "NLP is so cool",                   # from the JSON tweet
]

# a tiny taste of Stage 2 (just Lowercasing here, to show the handoff)
ready_for_cleaning = [doc.lower() for doc in collected]

print("STAGE 1 output (acquired):", collected)
print("STAGE 2 input (cleaning):", ready_for_cleaning)
print("\n-> from here, your full preprocessing pipeline takes over.")
```

```
STAGE 1 output (acquired): ['Natural Language Processing is fun!', 'Loved this product!', 'NLP is so cool']
STAGE 2 input (cleaning): ['natural language processing is fun!', 'loved this product!', 'nlp is so cool']
```

-> from here, your full preprocessing pipeline takes over.

Things every student should remember

- Data acquisition is **Step 0** — it happens *before* cleaning.
- The goal is always the same: **turn any source into a string (or list of strings)**.
- **CSV with pandas** is the format you'll meet most often in real datasets.
- **Built-in datasets** are the easiest way to practise — no messy loading.
- **Web pages bring HTML** — which is why your preprocessing removes tags.

Lab Exercise

1. Add two more rows to `reviews.csv`, re-read it, and print the new texts.
2. From `movie_reviews`, print the **number** of positive vs negative reviews.
3. Change the HTML string in Way 6 and confirm the tags are removed.
4. **Think:** which of the six sources gives the *messiest* text, and why?

